

## 27. Payment Gateway Integration (MakeMyTrip-style)

---

**Interviewer:** Different angle from building a gateway. You're **MakeMyTrip** — a travel booking product. You do NOT build your own card processor; you integrate an external **payment gateway** (Razorpay/Stripe/PayU). A user books a flight and pays. Design the **integration**: how your booking system talks to the gateway, keeps the booking and the payment in sync, and never ends up with "money taken but no ticket" or "ticket issued but no money." Go.

**Candidate:** This is the problem *most* engineers actually face — not building payment rails, but **safely orchestrating your business transaction with an external payment provider whose responses are asynchronous and unreliable**. The core risk is the **dual-write / two systems problem**: your booking DB and the payment provider must agree, but they're separate systems that can each fail independently. The tools: a **saga/state machine**, **idempotency**, **webhooks + polling**, and **reconciliation**. Let me clarify.

---

### 1. Clarifying the requirements

---

**Candidate:** 1. What are we buying — a flight/hotel with **limited inventory** we must hold during payment? (That couples payment to a seat/room hold, like our BookMyShow/Hotel designs.) 2. Do we redirect the user to the gateway's hosted page, or collect card details ourselves? (Hosted checkout keeps us out of PCI scope — I'll assume that.) 3. The gateway confirms payment via **synchronous redirect return** AND **async webhook** — do we treat the webhook as the source of truth? (We should.) 4. What's the failure we must absolutely prevent — **money debited but booking not confirmed**? Yes, that's the nightmare that generates angry customers and manual refunds.

**Interviewer:** Flights with a temporary fare/seat hold during checkout. Use the gateway's **hosted checkout** (redirect) so you're not handling cards. Treat the **webhook as the authoritative** payment signal, but the redirect return is a hint. Absolutely prevent "money taken, no ticket."

**Candidate:** Perfect. Requirements:

**Functional** - User selects a flight → we place a **hold** on the fare/seat → create an **order** → redirect to gateway → user pays → we confirm the booking and issue the ticket. - Handle all payment outcomes: success, failure, timeout, user-abandons, duplicate. - Auto-refund + release hold if payment succeeds but booking can't be fulfilled. - Idempotent throughout; reconcile with the gateway.

**Non-functional** - **No inconsistent end states** — every order ends as either (paid + ticketed) or (not-paid + released), never a stuck middle. This is the whole game. - **Durability** of order/payment state. - **Handle the external gateway being slow/flaky** as the normal case. - Reasonable latency, but correctness over speed on the money path.

---

## 2. The central problem — two systems, independent failures

---

**Candidate:** Let me name the trap before designing. We have **two separate systems**: our **Order/Booking DB** and the **external gateway**. Any step between them can fail:

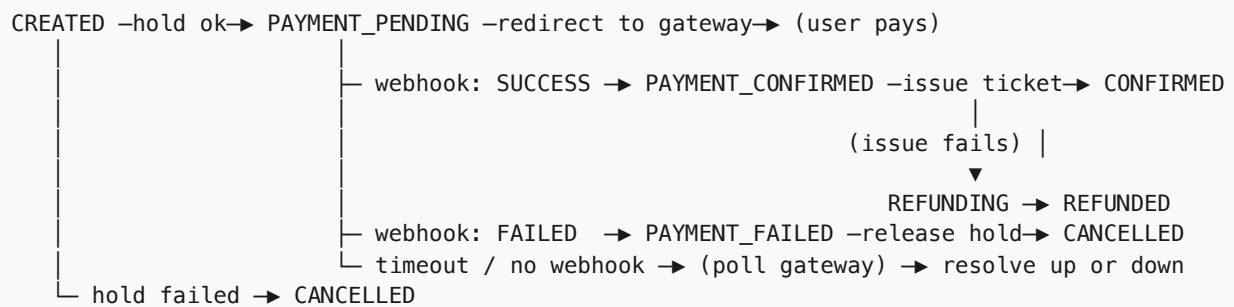
|                  |                           |                                           |
|------------------|---------------------------|-------------------------------------------|
| our DB says PAID | but gateway never charged | -> we ship a free ticket (revenue loss)   |
| gateway charged  | but our DB says UNPAID    | -> money taken, no ticket (the nightmare) |
| we hold a seat   | but user never pays       | -> inventory leaks                        |

You **cannot** make "charge the card" and "confirm the booking" one atomic transaction — they live in different systems. So we use a **saga**: a sequence of steps, each with a **compensating action**, driven by a **durable state machine**, and made safe with **idempotency** and **reconciliation**.

---

## 3. The order state machine (the backbone)

---



Every state is **PERSISTED** before the external call. No step is implicit.

The key insight: **we persist the intended state transition durably before each external action**, so a crash never loses track of where we are. On restart, a recovery process picks up any order stuck in a transient state and resolves it.

---

## 4. The happy-path flow (with the safety built in)

1. User picks flight
- ▼
2. HOLD fare/seat (Redis hold with TTL, like BookMyShow) → our inventory
- ▼
3. CREATE ORDER in DB, state=PAYMENT\_PENDING, generate order\_id + idempotency key
- ▼
4. CREATE payment intent at gateway (pass our order\_id as reference + idempotency key)
- ▼
5. REDIRECT user to gateway hosted checkout page
- ▼
6. User pays on gateway's page
- ▼
- 7a. Gateway REDIRECTS user back to us with a status (HINT – do not trust alone)
- 7b. Gateway sends us a WEBHOOK: payment.success (AUTHORITATIVE, signed)
- ▼
8. On webhook success (verify signature!): state → PAYMENT\_CONFIRMED
- ▼
9. ISSUE TICKET (call airline/inventory to confirm the held seat)
  - └ success → state=CONFIRMED, send confirmation
  - └ fail → state=REFUNDING → auto-refund via gateway → REFUNDED (release hold)

**Why webhook is authoritative, redirect is a hint:** the user's browser redirect can be lost (they close the tab, network drops, they navigate away) — but the webhook is a server-to-server call the gateway **retries** until we acknowledge. So we **never** confirm a booking solely because the browser came back "success"; we wait for (or poll for) the webhook. The redirect just lets us show a nice "processing..." page immediately.

## 5. Idempotency + webhook handling — the correctness core

**Interviewer:** Gateways send webhooks **multiple times** (at-least-once) and sometimes out of order. And users double-click "Pay." How do you stay correct?

**Candidate:** Idempotency on both directions:

**Outbound (our calls to the gateway):** every create-payment / refund call carries an **idempotency key** (usually derived from `order_id`). Retrying a create never makes two charges — the gateway dedups on the key.

**Inbound (webhooks from the gateway):**

- on webhook received:
1. VERIFY SIGNATURE (HMAC with shared secret) → reject forgeries
  2. dedup on gateway's event\_id (store processed event\_ids) → ignore repeats
  3. load order; apply transition **ONLY** if valid for current state (idempotent state machine)
    - e.g. success webhook when already CONFIRMED → no-op, just ACK 200
  4. ACK 200 quickly; do heavy work (ticket issue) async if needed

- **Signature verification** stops an attacker forging a "payment success."
- **Event dedup** makes repeated webhooks harmless.

- **State-guarded transitions** make out-of-order webhooks safe — a late "pending" webhook can't downgrade a CONFIRMED order.
- **ACK fast:** respond 200 immediately so the gateway stops retrying; if issuing the ticket is slow, do it async and let the state machine drive it.

## 6. The "no webhook arrived" case — polling + reconciliation

**Interviewer:** The webhook never comes (gateway hiccup, our endpoint was down). The user paid. Now what?

**Candidate:** Never rely on a single signal. Three overlapping mechanisms guarantee we resolve every order:

1. **Redirect hint** triggers an immediate **status poll**: when the user returns, we call the gateway's "get payment status for order\_id" API to check directly.
2. **Reconciliation poller**: a background job scans orders stuck in `PAYMENT_PENDING` past a threshold and **queries the gateway** for their real status, then drives the state machine (confirm + issue, or fail + release).
3. **Daily settlement reconciliation**: the gateway provides a settlement report; we compare it against our orders. Any "gateway charged, we show unpaid" is caught and repaired (issue the ticket or auto-refund). This is the ultimate backstop — same principle as a payment gateway reconciling with banks.

```
webhook (primary) }
redirect->poll    } → all feed the SAME idempotent state machine → terminal state
reconciliation job }      (whichever resolves first wins; rest are no-ops)
```

Because the state machine is idempotent, it doesn't matter which of the three resolves the order first — the others become no-ops.

## 7. Which datastore / tech — and why (the detailed part)

| Concern                                         | Choice                                       | Why this, and why not the alternative                                                                                                                                                                                                                                                          |
|-------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Orders + payment state (source of truth)</b> | <b>Relational ACID DB (PostgreSQL/MySQL)</b> | Order↔payment↔booking are related entities needing <b>atomic transactions and strong consistency</b> (persist state before each external call; guarded transitions). <i>Not eventual/NoSQL</i> — an inconsistent order state is money-taken-no-ticket. Volume is modest; ACID is the priority. |
| <b>Fare/seat hold</b>                           | <b>Redis key with TTL</b>                    | Fast atomic hold during checkout that auto-releases on abandonment (same pattern as BookMyShow/Hotel). <i>Not a DB lock held across a multi-minute human payment</i> — that ties up connections and inventory.                                                                                 |

| Concern                       | Choice                                                       | Why this, and why not the alternative                                                                                                                                                                                                                                                 |
|-------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Idempotency (in + out)</b> | <b>DB unique constraint on keys + processed event_id set</b> | Hard dedup for outbound calls and inbound webhooks. <i>Not best-effort</i> — a missed dedup double-charges or double-issues.                                                                                                                                                          |
| <b>Async orchestration</b>    | <b>Kafka (or a durable queue)</b>                            | Carries order events → ticket issuance, notifications, reconciliation, analytics; lets us ACK webhooks fast and do fulfillment async; replayable. <i>Not synchronous fulfillment inside the webhook handler</i> — a slow airline API would cause the gateway to time out and re-send. |
| <b>Webhook endpoint</b>       | <b>Idempotent, signature-verifying receiver + fast ACK</b>   | Security + at-least-once safety.                                                                                                                                                                                                                                                      |
| <b>Reconciliation store</b>   | <b>Same relational DB + settlement import job</b>            | Compare our ledger of orders vs the gateway's settlement file.                                                                                                                                                                                                                        |
| <b>Gateway abstraction</b>    | <b>Adapter layer per provider</b>                            | So we can support Razorpay/Stripe/PayU behind one internal interface and fail over between them.                                                                                                                                                                                      |

**Say-it-out-loud justification:** *"Because our booking and the external gateway are two systems that fail independently, I model the purchase as a saga on a strongly-consistent relational order DB, persisting each state transition before the external call so no crash loses track. Idempotency keys on outbound calls and signature-verified, deduped, state-guarded webhook handling keep us correct against retries and out-of-order delivery. The webhook is authoritative but I never trust a single signal — a status poll on redirect and a reconciliation job against the gateway's settlement file guarantee every order reaches a clean terminal state. Redis holds inventory during checkout; Kafka pushes fulfillment off the webhook path so we can ACK fast."*

## 8. Compensating actions (the saga's "undo")

Each forward step has a compensation, so a failure anywhere rolls back cleanly:

| Forward step   | If a later step fails, compensate with                |
|----------------|-------------------------------------------------------|
| Hold seat      | Release hold (TTL also auto-releases)                 |
| Charge payment | <b>Refund</b> payment (auto, via gateway, idempotent) |
| Issue ticket   | Cancel ticket with airline                            |

The classic case: **payment succeeds but ticket issuance fails** (airline API down, seat gone). We do **not** leave the customer charged — the saga transitions to `REFUNDING` and issues an automatic refund, releasing the hold. Money-taken-no-ticket becomes money-returned-no-ticket, which is acceptable and automatic rather than a manual support ticket.

## 9. Consistency, failure modes

- **Consistency:** strong within our order DB; the *cross-system* consistency (us ↔ gateway) is **eventual but guaranteed-to-converge** via webhook + poll + reconciliation. We accept a brief window where the user paid and we're still confirming, shown as "processing your booking."
- **Availability stance:** if we can't record the order durably, **don't take payment**. If payment succeeded but we can't fulfill, **auto-refund**. Always fail toward "customer not left charged without value."

| Failure                        | What happens                   | Mitigation                                                                |
|--------------------------------|--------------------------------|---------------------------------------------------------------------------|
| Webhook lost / endpoint down   | Order stuck<br>PAYMENT_PENDING | Redirect-poll + reconciliation job query the gateway and resolve          |
| Duplicate/out-of-order webhook | Risk of double-processing      | Signature + event dedup + state-guarded transitions                       |
| Payment ok, ticket issue fails | Money taken, no seat           | Saga compensates → auto-refund + release hold                             |
| User abandons after hold       | Inventory leak                 | Hold TTL auto-releases; order expires to CANCELLED                        |
| Our service crashes mid-flow   | Partial state                  | State persisted before each step; recovery job resumes stuck orders       |
| Gateway down at checkout       | Can't pay                      | Show retry; hold TTL protects inventory; optional failover to alt gateway |

## 10. Follow-up curveballs

**Interviewer:** User clicks "Pay" twice and you create two payment intents. Now two charges?

**Candidate:** No — the create-payment call is **idempotent on `order_id`**, so the second attempt returns the *same* payment intent, not a new one. And our order is already in `PAYMENT_PENDING`; we don't create a second order for the same cart/idempotency key. One order, one payment intent, at most one charge.

**Interviewer:** How do you support multiple gateways and route between them?

**Candidate:** A **gateway adapter layer** — one internal interface ( `createPayment`, `refund`, `getStatus`, `verifyWebhook` ) with per-provider implementations. A routing policy picks a gateway by cost, success-rate, currency, or region, and can **fail over** if one is down. Each adapter normalizes that provider's webhook format into our internal events. The state machine and reconciliation stay provider-agnostic.

**Interviewer:** The user's money is debited but the gateway shows "failed" to you (bank settled late). What happens?

**Candidate:** This surfaces in **reconciliation** against the settlement file: the gateway/ bank shows a successful settlement our order marked failed. The recon job flags it and either auto-issues the ticket (if inventory is still available) or auto-refunds the customer. This is exactly why the settlement reconciliation backstop exists — online signals can be wrong or late, the settlement file is authoritative for what money actually moved.

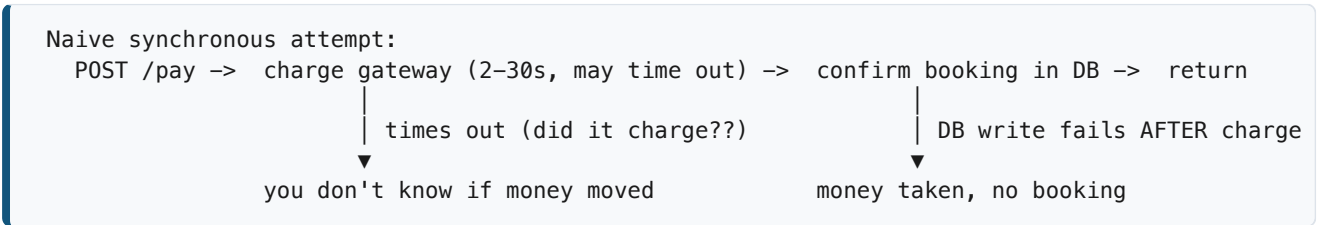
**Interviewer:** Multi-currency, user in India paying for a US hotel?

**Candidate:** The gateway handles currency conversion/settlement; we store the order in the charged currency + our settlement currency, and pick a gateway/acquirer that supports the corridor. Data residency (India's RBI rules, GDPR) may require region-pinned storage of payment references — route and store per regulation.

## 11. Deep-dive: "Why not do the booking and payment in one synchronous call?"

**Interviewer:** Why all this saga machinery? Why not just: user clicks pay → your server charges the card and confirms the booking in one request, return success? Simple.

**Candidate:** Because "charge the card" happens in the **external gateway** and "confirm the booking" happens in **your DB** — two systems that fail independently. A single synchronous call can't make them atomic, and every failure point leaves you in a bad state.



| Concern                             | Synchronous charge-then-confirm                                                   | Saga (state machine + async)                                      |
|-------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Gateway is slow (2-30s)             | User's request blocks; connections pile up; timeouts                              | User redirected to gateway; your server isn't holding a thread    |
| Timeout mid-charge                  | You don't know if the card was charged → guess wrong = double charge or lost sale | Webhook + status poll resolve it deterministically later          |
| DB fails after charge succeeds      | Money taken, no ticket, no record to recover from                                 | State was persisted <i>before</i> the charge; recovery resumes it |
| Retry after failure                 | Naive retry re-charges                                                            | Idempotency key makes retries safe                                |
| Inventory held during human payment | A DB transaction/lock held for minutes                                            | Redis hold with TTL, no long-lived lock                           |

The synchronous version isn't just slower — it's **unrecoverable**, because it has no durable record of "a charge is in flight" to reconcile against when something fails in the gap.

**Rule of thumb:** any time a business action spans your system *and* an external one, model it as a saga with persisted state and compensations — never a single synchronous call that assumes both succeed together.

## 12. Deep-dive: the redirect-says-success-but-no-webhook race (and double webhooks)

**Interviewer:** The user pays, the gateway redirects their browser back to you with "success," but the webhook hasn't arrived yet — or never arrives. Meanwhile the same webhook sometimes arrives twice. Walk me through keeping this correct.

**Candidate:** This is the crux of gateway integration: **two signals about the same payment, arriving in any order, any number of times, some not at all.** The redirect is a hint from an untrusted browser; the webhook is the authoritative server-to-server truth — but it's at-least-once and can be late or lost.

Case A — redirect arrives first, webhook lags:

t0 user pays on gateway  
t1 browser redirect -> your /return?status=success (HINT — do NOT confirm on this alone)  
t2 (webhook still in flight)  
t3 webhook payment.success arrives (AUTHORITATIVE — now confirm)

Case B — webhook never arrives (gateway hiccup / your endpoint was down):

redirect said success, but no webhook -> order stuck PENDING forever if you wait passively

Case C — webhook arrives TWICE (at-least-once delivery):

naive handler confirms twice / issues two tickets

Ranked defenses, combined:

1. **Never confirm on the redirect alone.** On `/return`, show a "processing..." page and **trigger an immediate status poll** — call the gateway's "get payment status for order\_id" API to learn the real outcome directly. This resolves Case A instantly without waiting for the webhook, and covers Case B's start.
2. **Idempotent, state-guarded webhook handler** (Case C). Verify the signature, dedup on the gateway's `event_id`, and apply the state transition **only if valid for the current state**. A second `success` webhook for an already- `CONFIRMED` order is a no-op that just returns 200. So duplicates and out-of-order arrivals are harmless.
3. **Reconciliation poller** (Case B backstop). A job sweeps orders stuck in `PAYMENT_PENDING` past a threshold and queries the gateway for their true status, then drives the state machine — so a lost webhook never strands an order.
4. **All three feed one idempotent state machine.** Redirect-poll, webhook, and reconciliation are three roads to the same transition; whichever arrives first wins, the rest are no-ops. That's what makes "any order, any count, some missing" safe.

redirect -> status poll  
webhook (authoritative)  
reconciliation job

→ same idempotent, state-guarded transition → CONFIRMED (once)

**What to say out loud:** *"I treat the webhook as authoritative but never trust a single signal. The redirect only triggers a status poll — I confirm on the gateway's actual answer, not the browser. The webhook handler is idempotent and state-guarded, so duplicates and out-of-order deliveries are no-ops. And a reconciliation poller resolves any order whose webhook never came. Three signals, one idempotent state machine — so the order always reaches the right terminal state exactly once, no matter what arrives or in what order."*

## Summary — the mental model

Integrating a payment gateway is the **"two systems that fail independently must always agree"** problem. You cannot make the charge and the booking one atomic transaction, so you use a **saga**: a durable, idempotent **state machine** on a strongly-consistent order DB, persisting each transition **before** the external call. Make the gateway the **authoritative** signal via **signed, deduped, state-guarded webhooks**, but never trust a single signal — back it with a **redirect-triggered status poll** and a **reconciliation job against the settlement file** so every order converges to a clean terminal state. Hold inventory in Redis with a TTL, push fulfillment async via Kafka so you can ACK webhooks fast, and give every forward step a **compensating action** (auto-refund, release hold) so the system always fails toward "the customer is never charged without getting value." That's how MakeMyTrip-style products avoid the money-taken-no-ticket nightmare.